

PHP - Strings

They are sequences of characters, like "PHP supports string operations".

NOTE – Built-in string functions is given in function reference [PHP String Functions](#) 

Following are valid examples of string

```
$string_1 = "This is a string in double quotes";  
$string_2 = "This is a somewhat longer, singly quoted string";  
$string_39 = "This string has thirty-nine characters";  
$string_0 = ""; // a string with zero characters
```

Singly quoted strings are treated almost literally, whereas doubly quoted strings replace variables with their values as well as specially interpreting certain character sequences.

[Live Demo](#)

```
<?php  
    $variable = "name";  
    $literally = 'My $variable will not print!\n';  
  
    print($literally);  
    print "<br />";  
  
    $literally = "My $variable will print!\n";  
  
    print($literally);  
?>
```

This will produce the following result –

```
My $variable will not print!\n  
My name will print!\n
```

There are no artificial limits on string length - within the bounds of available memory, you ought to be able to make arbitrarily long strings.

Strings that are delimited by double quotes (as in "this") are preprocessed in both the following two ways by PHP –

-  Certain character sequences beginning with backslash (\) are replaced with special characters
-  Variable names (starting with \$) are replaced with string representations of their values.

The escape-sequence replacements are –

-  \n is replaced by the newline character
-  \r is replaced by the carriage-return character

- `\t` is replaced by the tab character
- `\$` is replaced by the dollar sign itself (`$`)
- `\"` is replaced by a single double-quote (`"`)
- `\\` is replaced by a single backslash (`\`)

String Concatenation Operator

To concatenate two string variables together, use the dot (`.`) operator –

[Live Demo](#)

```
<?php
$string1="Hello World";
$string2="1234";

echo $string1 . " " . $string2;
?>
```

This will produce the following result –

```
Hello World 1234
```

If we look at the code above you see that we used the concatenation operator two times. This is because we had to insert a third string.

Between the two string variables we added a string with a single character, an empty space, to separate the two variables.

Using the `strlen()` function

The `strlen()` function is used to find the length of a string.

Let's find the length of our string "Hello world!" –

[Live Demo](#)

```
<?php
echo strlen("Hello world!");
?>
```

This will produce the following result –

```
12
```

The length of a string is often used in loops or other functions, when it is important to know when the string ends. (i.e. in a loop, we would want to stop the loop after the last character in the string)

Using the `strpos()` function

The `strpos()` function is used to search for a string or character within a string.

If a match is found in the string, this function will return the position of the first match. If no match is found, it will return FALSE.

Let's see if we can find the string "world" in our string –

[Live Demo](#)

```
<?php
    echo strpos("Hello world!","world");
?>
```

This will produce the following result –

6

As you see the position of the string "world" in our string is position 6. The reason that it is 6, and not 7, is that the first position in the string is 0, and not 1.